

Git & GitHub: branch, commit, PR, merge, conflicts

Головна ідея: гілка (branch) дозволяє вносити зміни, не чіпаючи основний код. Коли зміни готові -- їх оформлюють як Pull Request (PR), команда переглядає код, і після схвалення відбувається злиття (merge) у основну гілку. Якщо дві гілки міняли один і той самий рядок -- виникає конфлікт (conflict), який треба вирішити вручну.

1. Branch (гілка)

Гілка -- окрема, незалежна лінія розробки. Дозволяє експериментувати чи виправляти щось, не ризикуючи стабільністю основного коду (зазвичай гілки `main`).

Перегляд гілок

```
git branch          # локальні гілки (поточна позначена *)
git branch -r       # лише віддалені гілки (origin/...)
git branch -a       # усі гілки: локальні + віддалені
```

Створення та перехід: git switch і git checkout

```
git switch -c page-header  # створити нову гілку і одразу перейти на неї
git switch page-header     # перейти на існуючу гілку
```

`git checkout` -- старіша, універсальніша команда. До появи `git switch` (Git 2.23+) вона виконувала одразу декілька різних функцій: перехід між гілками, створення нової гілки та відновлення файлів зі стану коміту.

```
git checkout page-header  # перейти на існуючу гілку (старий спосіб)
git checkout -b page-header  # створити нову гілку і перейти на неї (аналог switch -c)
git checkout -- index.html  # відкинути незакомічені зміни у файлі (повернути до останнього коміту)
```

Аспект	git checkout	git switch
Призначення	універсальна: гілки + файли + коміти	лише перехід/створення гілок
Ризик помилки	вищий -- легко переплутати гілку з файлом і випадково втратити зміни	нижчий -- команда робить тільки одну річ
Синтаксис створення гілки	<code>git checkout -b name</code>	<code>git switch -c name</code>
Рекомендація	використовувати для відновлення файлів (<code>checkout -- file</code>)	використовувати для роботи з гілками (сучасний, безпечніший варіант)

Висновок: `git switch` створили спеціально, щоб розділити функціонал `checkout` і зробити роботу з гілками очевиднішою та безпечнішою. Для нових проєктів варто використовувати `switch` для гілок і залишити `checkout` тільки для відновлення файлів.

Перед створенням нової гілки перевір `git status` -- усі зміни мають бути закомічені ("nothing to commit, working tree clean"), інакше спершу зроби коміт.

Практика правильних назв гілок

- коротко й конкретно відображає задачу (не `fix` , а `fix-login-bug`)
- слова розділяти дефісом `-` або підкресленням `_`
- писати у нижньому регістрі

Підключення до віддаленого репозиторію

```
git remote add origin git@github.com:username/repo.git  # прив'язати локальний репо до віддаленого
git remote -v                                           # перевірити прив'язку (fetch/push URL)
git remote set-url origin git@github.com:username/new.git  # змінити URL, якщо помилились
```

Потрібно один раз при переході з локального `git init` на роботу з командою -- без цього `git push / git pull` не знають, куди звертатись.

Робота з віддаленими гілками

Команда	Що робить
<code>git fetch</code>	завантажує зміни з віддаленого репозиторію, але НЕ застосовує їх до робочого каталогу
<code>git pull</code>	завантажує і одразу застосовує зміни з усіх гілок
<code>git pull origin footer</code>	завантажує й застосовує зміни лише з віддаленої гілки <code>footer</code>

2. Commit (коміт)

Коміт -- "знімок" стану всього проєкту в певний момент. Кожна гілка має свою окрему історію комітів, ізольовану від інших гілок до моменту злиття.

```
git add .          # додати зміни в індекс (staging area)
git commit -m "Fix typo in README"  # зафіксувати зміни у поточній гілці
```

Правила хороших повідомлень коміту

- пиши англійською;
- наказова форма: `Add` , `Fix` , `Update` (не `Added` , `Fixed`);
- лаконічно, але зрозуміло -- що саме змінено.

```
git log --oneline  # компактна історія комітів поточної гілки
```

3. Робота в команді: колаборанти та захист гілок

Додавання колаборанта до репозиторію

1. Settings -> Access -> Collaborators -> Add people

- 2. Вказати нік або пошту користувача GitHub
- 3. Коллаборант отримує лист-запрошення на пошту, має підтвердити перехід за посиланням

Командний репозиторій не з'являється у "власних" -- його шукати через аватар -> Settings -> Repositories (там показано і власні, і командні).

Захист гілки main (branch protection)

Щоб заборонити прямі зміни в `main` і вимагати перевірку коду:

- 1. Settings -> Branches -> Add branch protection rule
- 2. У полі Branch name pattern вказати `main`
- 3. Обрати **Require a pull request before merging** (+ Require approvals -- мінімум 1 погодження)

Хто змінював код: git blame

```
git blame index.html      # показує, хто і в якому коміті востаннє змінив кожен рядок файлу
git blame -L 10,20 index.html  # тільки рядки з 10 по 20
```

Корисно в команді, коли треба зрозуміти, чому рядок написаний саме так, і до кого звернутись із питанням -- перед тим, як міняти чужий код.

4. Pull Request (PR)

PR -- запит на злиття змін з однієї гілки в іншу (зазвичай у `main`). Дозволяє команді переглянути код перед злиттям.

Створення PR (fork -> branch -> PR)

- 1. **Fork** -- копія чужого репозиторію у свій акаунт (кнопка Fork)
- 2. Клонувати форк собі: `git clone git@github.com:твій-нік/repo.git`
- 3. Створити гілку: `git switch -c fix-typo-readme`
- 4. Внести зміни -> `git add .` -> `git commit -m "Fix typo"`
- 5. Відправити гілку: `git push origin fix-typo-readme`
- 6. На GitHub: вкладка Pull requests -> New pull request
- 7. Обрати source branch (звідки зливаємо) і target branch (куди зливаємо, зазвичай `main`)
- 8. Вписати назву й опис PR -> Create pull request

Інтерфейс Pull Request

Вкладка / елемент	Призначення
Commits	усі коміти, включені у цей PR
Files changed	у яких саме файлах відбулися зміни (додано/видалено рядки)
Reviewers	призначення рецензента з команди
Коментарі	можна залишити коментар до всього PR або під конкретним рядком коду
Close pull request	скасувати запит на злиття (доступно автору й власнику репо)

Прийняття PR (рев'ю)

- 1. Pull Requests -> обрати потрібний PR
- 2. Add your review -> переглянути код, залишити коментарі під рядками (Start your review)
- 3. Finish your review -> обрати: **Approve** / **Request changes** / **Comment**
- 4. Якщо Approve -- стає активною кнопка **Merge pull request** -> Confirm merge
- 5. Після злиття можна одразу натиснути **Delete branch**

Історію закритих і прийнятих PR дивись у вкладці Pull Requests -> Closed.

5. Merge (злиття)

Через PR на GitHub (рекомендовано в команді)

Merge pull request -> Confirm merge -- GitHub зробить це сам, якщо конфліктів немає.

Локально через термінал

```
git switch main
git pull origin main      # оновити main до останньої версії
git merge page-header     # злити гілку page-header у main
git push origin main      # відправити оновлений main на GitHub
```

Злиття впливає лише на поточну гілку (`main`). Гілка `page-header` залишається без змін.

Види злиття (merge types)

Коли виконуєш `git merge`, Git обирає один із двох підходів залежно від того, чи розходилась історія гілок.

Тип	Коли застосовується	Як виглядає в git history graph
Fast-forward (за замовчуванням)	Основна гілка (<code>main</code>) не отримувала нових комітів, поки ти працював у <code>page-header</code> . Git просто "пересуває" вказівник <code>main</code> вперед, до останнього коміту гілки.	Пряма лінія без розгалужень -- виглядає так, ніби всі коміти робились одразу в <code>main</code> . Окремого merge-коміту не створюється.
Merge commit (звичайний, non-fast-forward)	Основна гілка встигла отримати нові коміти, поки ти працював у своїй гілці (історія розійшлась). Git не може просто "пересунути" вказівник і створює новий коміт злиття із двома батьками.	Видно точку розгалуження і точку злиття -- дерево має "ромбовидну" форму: дві лінії розходяться і знову сходяться в одному коміті.
No fast-forward (<code>--no-ff</code>)	Так само, як fast-forward (лінійна історія можлива), але ти свідомо просиш Git все одно створити merge-коміт.	Так само як merge commit -- завжди видно "ромб" і окремий коміт злиття, навіть якщо технічно можна було б просто пересунути вказівник.

```
git merge page-header      # fast-forward, якщо можливо; інакше -- звичайний merge commit
git merge --no-ff page-header  # завжди створює merge-коміт, навіть якщо можливий fast-forward
```

Навіщо потрібен --no-ff : зберігає в історії явний слід, що гілка `page-header` існувала і була окремою фічею/фіксом -- зручно для аудиту та для команд, де важливо бачити межі кожної задачі в `git log --graph` . Fast-forward натомість дає "чистішу", лінійну історію, але приховує факт існування гілки.

Візуальний перегляд історії: gitk

```
gitk          # відкриває графічне вікно з деревом комітів поточного репозиторію
gitk --all    # показати історію всіх гілок (локальних і віддалених), не тільки поточної
```

`gitk` -- вбудований у Git графічний інструмент (встановлюється разом із Git). Показує те саме дерево комітів, що й `git log --graph` , але у вигляді клікабельного вікна: видно гілки, злиття, автора й діф кожного коміту. Зручно, коли текстовий графік у терміналі важко читати.

Видалення гілки після злиття

```
git branch -d page-header    # видалити локально (тільки якщо гілку вже змержено)
git branch -D page-header    # примусове видалення (навіть не змерженої)
git push origin --delete page-header    # видалити віддалену гілку
```

Перед видаленням віддаленої гілки в команді -- попередь колег, щоб усі встигли забрати потрібні зміни.

6. Merge conflicts (конфлікти злиття)

Виникає, коли дві гілки змінили один і той самий рядок файлу -- Git не може сам вирішити, яку версію лишити.

Як виглядає конфлікт у файлі

```
<<<<<<< HEAD
Код з поточної гілки (наприклад, header)
=====
Код з гілки, яку зливаємо (наприклад, footer)
>>>>>>> footer
```

Варіант А -- вирішення локально (термінал / VSCode)

- `git merge footer` -> Git видасть: `CONFLICT (content): Merge conflict in index.html`
- `git status` покаже всі конфліктні файли (у VSCode вони підсвічені червоним)
- Відкрити файл, вручну обрати потрібний варіант (або скористатись кнопками VSCode: Accept Current / Accept Incoming / Accept Both)
- Видалити маркери `<<<<<<<` , `=====` , `>>>>>>>`
- `git add .` -> `git commit -m "Resolve merge conflict in index.html"`
- Перевірити результат: `git log`

Варіант Б -- вирішення прямо на GitHub (для простих випадків)

- У PR з'явиться попередження про конфлікт -> натиснути **Resolve conflicts**
- У вебредакторі GitHub вручну виправити конфліктні ділянки, видаливши маркери
- Натиснути **Mark as resolved** -> **Commit changes**
- Повернутись у PR -- повідомлення про конфлікт зникне, можна робити Merge

Рекомендований підхід в команді: краще повернути PR автору змін для локального вирішення конфлікту (він найкраще знає контекст свого коду) -- залишити коментар і позначити Request changes, а не виправляти чужий конфлікт самостійно на GitHub.

7. Список найбільш використовуваних команд

Команда	Призначення
<code>git init</code>	ініціалізувати новий локальний репозиторій
<code>git clone <url></code>	склонувати віддалений репозиторій собі на комп'ютер
<code>git status</code>	показати поточний стан репозиторію (staged/unstaged/untracked файли)
<code>git add <file> / .</code>	додати файл(и) у проміжну область (staging area)
<code>git commit -m "..."</code>	зафіксувати зміни у поточній гілці
<code>git commit --amend</code>	виправити останній (ще не запущений) коміт
<code>git log</code> / <code>git log --oneline</code>	переглянути історію комітів
<code>git log --graph</code>	переглянути історію комітів у вигляді дерева/графа гілок
<code>git branch</code>	список локальних гілок
<code>git branch -a</code>	список усіх гілок (локальні + віддалені)
<code>git switch -c <name></code>	створити нову гілку і перейти на неї
<code>git switch <name></code>	перейти на існуючу гілку
<code>git checkout -b <name></code>	створити нову гілку і перейти (старіший аналог switch -c)
<code>git branch -d / -D <name></code>	видалити локальну гілку (звичайно / примусово)
<code>git merge <branch></code>	злити вказану гілку в поточну
<code>git merge --no-ff <branch></code>	злити із примусовим створенням merge-коміту
<code>git fetch</code>	завантажити зміни з віддаленого репо без застосування
<code>git pull</code>	завантажити і одразу застосувати зміни з віддаленого репо
<code>git push</code>	відправити локальні коміти у віддалений репозиторій
<code>git push origin --delete <branch></code>	видалити віддалену гілку
<code>git remote add origin <url></code>	прив'язати локальний репозиторій до віддаленого

<code>git remote set-url origin <url></code>	змінити URL віддаленого репозиторію
<code>git remote -v</code>	показати список і URL підключених віддалених репозиторіїв
<code>git stash</code> / <code>git stash pop</code>	тимчасово відкласти незакомічені зміни / повернути їх
<code>git reset <file></code>	прибрати файл з індексу, зберігши зміни у робочому каталозі
<code>git rm --cached <file></code>	прибрати файл з відстеження Git, залишивши на диску
<code>git diff</code>	показати різницю між робочим каталогом і останнім комітом/індексом
<code>git blame <file></code>	хто і в якому коміті востаннє змінив кожен рядок файлу
<code>gitk</code> / <code>gitk --all</code>	графічний перегляд дерева комітів (поточна гілка / усі гілки)